Secrets Management

A secret is any value that, if exposed, would allow unauthorized access to a system or data. This includes: passwords, API keys, tokens, private keys, certificates, connection strings containing passwords, and PGP/GPG private keys used for DC feed decryption. It does not include: public keys, environment identifiers, non-sensitive URLs, or feature flag keys (which are designed to be readable).

The Reference Pattern

Application code and configuration must never contain a credential value. It contains only a secret reference identifying that the secret store resolves to the actual value at runtime. The secret provider is the only system that ever holds the plaintext credential.

Example

Code

"DbSecretRef": "cnp/prod/ut/db-connection"//

resolves at runtime ⟶

Secret Store

cnp/prod/ut/db-connection→ "Server=db;Password=Sup3rS3cr3t!"Access controlled · audited · rotatabl

Secret Consumption

```
// ISecretProvider — the only interface application code uses
public interface ISecretProvider {
  Task<string> GetSecretAsync(string secretRef);
  Task<T> GetSecretAsync<T>(string secretRef); // deserializes JSON secrets
}

// DC feed adapter — never holds a credential, only a reference
public class SftpDcFeedAdapter {
  private readonly ISecretProvider _secrets;
  private readonly string _credentialRef; // e.g. "cnp/prod/ut/dc-sftp"

  public async Task ConnectAsync() {
    var creds = await _secrets.GetSecretAsync<SftpCredential>(_credentialRef);
    // creds.Host, creds.Username, creds.Password resolved here — never stored
  }
}
```

The application code calls ISecretProvider so it has no knowledge of which secret store backs it. In production this resolves to whatever managed secret service is selected. In local development it can resolve to environment variables or a local vault. In CI/CD it may resolve to injected pipeline secrets.

Secret Categories

Category	Examples	Rotation frequency	Scope
Database credentials	cnp_global read/write password, cnp_ops per-state read/write passwords	90 days (automated)	Per environment (dev/staging/prod) + per tenant for cnp_ops
DC feed credentials	Per-state SFTP host/username/password, PGP private keys for encrypted feeds, REST API tokens for Medicaid/SNAP endpoints	As required by state agency (typically annually, sometimes quarterly)	Per tenant, per feed type
OIDC client secrets	Client secret for each OIDC provider registered with the auth broker	180 days or on compromise	Per OIDC provider registration
External service API keys	LaunchDarkly SDK key, FNS policy feed API token, Ed-Fi endpoint credentials, Snowflake service account credentials	90–180 days	Per environment
Signing keys	JWT signing key for session tokens, webhook signature keys	180 days; graceful rotation (old key valid for overlap period)	Per environment

	for outbound DC feed webhooks		
Encryption keys	PII field encryption keys for sensitive data at rest (student identifiers, addresses)	Annual; requires re-encryption of existing data on rotation	Per tenant
Infrastructure credentials	Container registry credentials, deployment pipeline service accounts	90 days	Per environment

⚠ We need to partner with infosec on rotation frequency

DC feed credentials

Unlike platform-managed credentials that the system rotates automatically, DC feed credentials are issued by external parties (state benefit agencies). When a client rotates their SFTP password, their IT contact would need to notify us. The team updates the secret in the secret store via the DC Feed Adapter API which stores the credential reference, not the value directly and the next nightly load uses the new credential automatically. No deployment. No code change.

Secret Naming Convention... Keep it secret, keep it safe

Secret references follow a hierarchical naming scheme that encodes environment, tenant, and purpose. This makes access control policies writable at the path level so a service running in the tenant context can be granted access to `cnp/prod/ut/*` without enumerating individual secrets.

```
-- Naming pattern:  cnp / {environment} / {tenant-or-global} / {category} / {name}

cnp/prod/global/db/cnp-global-rw          -- shared database read/write
cnp/prod/ut/db/cnp-ops-rw                 -- Utah cnp_ops read/write
cnp/prod/ut/dc-feed/snap-sftp             -- Utah SNAP SFTP credentials
cnp/prod/ut/dc-feed/medicaid-api-token    -- Utah Medicaid REST token
cnp/prod/me/dc-feed/snap-sftp             -- Maine SNAP SFTP credentials
cnp/prod/global/oidc/broker-signing-key   -- OIDC JWT signing key
cnp/prod/global/external/launchdarkly-sdk -- LaunchDarkly SDK key
cnp/prod/global/external/snowflake-svc    -- Snowflake service account
cnp/staging/ut/db/cnp-ops-rw             -- staging environment, separate path
```

The state application service needs access to thier own secrets and the global shared secrets. Its access policy is expressed as two path prefixes: `cnp/prod/ut/*` and `cnp/prod/global/*`. It cannot access another state's secrets (`cnp/prod/me/*`) because it has no policy for that path. This is per-tenant isolation enforced by the secret store's own access control so there's no application-level filtering.

Access Control

Access to secrets follows least-privilege principles. Each service identity gets access only to the specific secret paths it needs. No service has blanket access to all secrets.

Identity	Paths accessible	Permissions
Application service (per tenant)	`cnp/prod/{tenant}/*` and `cnp/prod/global/*`	Read only. Cannot create, update, or delete secrets.
DC Feed Adapter service	`cnp/prod/{tenant}/dc-feed/*`	Read only. Scoped to the tenant it's running for.
Onshore config admin (human)	`cnp/prod/{assigned-tenants}/dc-feed/*`	Read, write, rotate. Only for DC feed credentials. Cannot access database or signing key paths.
Platform operations team	`cnp/prod/global/*`	Read, write, rotate for platform-level secrets. No access to per-tenant paths.
Offshore engineers	`cnp/dev/*` and `cnp/staging/*`	Full access to non-production environments only. Zero access to `cnp/prod/*`.

CI/CD pipeline	Specific deployment secrets only, scoped to the environment being deployed to	Read only, short-lived credentials injected per pipeline run.

Non-US Persons Never have Production Secret Access

Offshore builds the ISecretProvider implementation and tests against dev/staging secrets. Production secret paths are inaccessible to offshore identities at the access control level. This satisfies both the offshore/onshore separation principle and the regulatory requirement that state agency data is protected from unauthorized access.

Rotation

Rotation is the process of replacing a live credential with a new one without service interruption. There are two rotation models depending on whether the credential can be changed unilaterally by the platform or requires coordination with an external party.

Platform-managed rotation (automated)

For credentials the platform controls (database passwords, JWT signing keys, internal service tokens) rotation is automated on a schedule. The secret store generates a new credential, updates the stored value, and notifies the application to refresh its cached copy. The application must tolerate a brief overlap period where both the old and new credential are valid, to avoid dropping in-flight requests.

Rotation is the process of replacing a live credential with a new one without service interruption. There are two rotation models depending on whether the credential can be changed unilaterally by the platform or requires coordination with an external party.

Externally-managed rotation (coordinated)

For DC feed credentials issued by state benefit agencies, the platform doesn't control when rotation happens — the state agency does. The process is:

1. State agency notifies LINQ that a credential is being rotated (at minimum 24 hours notice)
2. Onshore config team receives the new credential through a secure out-of-band channel (not email)
3. Onshore updates the secret in the secret store via the DC Feed Adapter API
4. The next nightly DC load picks up the new credential automatically via ISecretProvider
5. Onshore confirms the load succeeded; state agency retires the old credential

The key outcome is **no deployment required for any rotation**.

Secret Lifecycle

1 Create

Secret is registered with a reference path, initial value, and metadata (rotation policy, access policy, owner). Never stored anywhere else.

2 Distribute

Services receive the reference path (safe to store). They call ISecretProvider at runtime to resolve the value. The value is never persisted by the application.

3 Rotate

New value is written to the store. Application cache is invalidated. Old value remains valid through the overlap window, then expires. Full audit log entry created.

4 Revoke

On suspected compromise: immediate invalidation of current value, forced rotation, audit log flagged as incident. Service outage is accepted over continued exposure.

5 Retire

When a service is decommissioned or a state offboards, secrets are deleted with a retention hold (30 days) before permanent removal. Deletion is audited.

Example

Minnesota DC feed onboarding (new state)

Minnrsota is being onboarded. Their SNAP SFTP credentials are provided during the configuration design session. Configuration team registers the credential via DC Feed Adapter API

```
POST /dc-feeds/mn
{
  "feedType": "SFTP",
  "programCode": "SNAP",
  "host": "sftp.dhs.mn.gov",
  "username": "linq_dc_prod",
  "password": "[provided securely out-of-band]",
  "pgpKeyRef": "cnp/prod/mn/dc-feed/snap-pgp-key"  // PGP key registered separately
}

```

- API stores the credential in the secret store at: cnp/prod/mn/dc-feed/snap-sftp
- Returns: { "secretRef": "cnp/prod/mn/dc-feed/snap-sftp", "feedId": "feed-mn-snap-001" }
- The password is never stored in any application database.
- The API response contains only the reference path.

Audit Trail

Every operation on a secret is logged: creation, reads, updates, rotation, revocation, and deletion. The audit log is write-only — it cannot be modified or deleted, even by the platform operations team. This is the evidence trail for any security incident investigation and for regulatory audits.

Event	Logging	Retention
Secret created	Reference path, creating identity, timestamp, rotation policy set	Permanent
Secret read	Reference path, accessing service identity, timestamp. Not the value itself.	90 days
Secret updated / rotated	Reference path, updating identity, timestamp, previous version reference	Permanent
Secret revoked	Reference path, revoking identity, timestamp, reason code, incident flag if applicable	Permanent
Access denied	Reference path, requesting identity, timestamp, denial reason	90 days
Secret deleted	Reference path, deleting identity, timestamp, retention hold expiry	Permanent

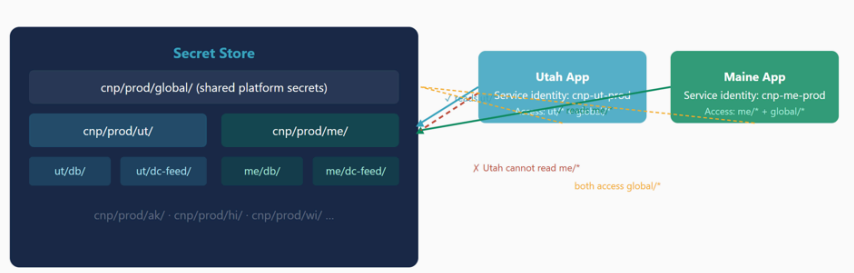
Secret Reads 📚

Secret read events are high-volume — log them separately

Secret reads may generate numerous log entries per day. These should be written to a high-volume append-only log, not to the same store as creation/rotation/revocation events. Rotation and revocation events are low-volume and must be immediately queryable by the security team. Read events should feed into anomaly detection (unusually high read rate on a specific secret may indicate credential stuffing). 🦃

Per-Tenant Isolation

The CNP platform's database-per-tenant architecture has a direct parallel in secrets management. Each state's credentials are isolated at the path level. A Utah application instance cannot read Maine's secrets even if both are running in the same environment.



This mirrors the database architecture. Just as cnp_ops has a separate schema per state and no TenantId column is needed, the secret store has a separate path prefix per state and no application-level tenant filtering is needed. The isolation is enforced by the infrastructure, not by application code.

Governance

These are the rules that govern secrets management across all environments, all phases, and all team structures.

Rule	Reason
No secret value ever appears in source code, configuration files, or container images	Source code is visible to too many people and persists in git history even after deletion
No secret value ever appears in application logs	Logs are widely distributed, retained for long periods, and often accessible to more people than production systems
No secret value is ever stored in an application database	Application databases are backed up, replicated, and accessible to DBAs — none of whom need credential access
Offshore engineers have zero access to production secrets	Offshore teams build against dev/staging; production secret access is an onshore operations function
Every secret has a rotation policy — "never rotate" is not a valid policy	Long-lived unrotated credentials are the most common vector for credential-based breaches
Secret values are transmitted only via the secret store API — never via email, Slack, or ticketing systems	Messaging systems are not designed for credential security and are frequently compromised
Suspected credential compromise triggers immediate revocation — service interruption is acceptable, continued exposure is not	A brief outage is recoverable; a data breach involving state agency PII is not
Application code calls ISecretProvider — it never constructs a secret store API call directly	The abstraction ensures consistent access control, caching, and audit logging regardless of which service is calling